

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 1 — Fondamenti degli LLM

Token e finestra di contesto

Quanti token?

Come un LLM vede il testo, e quanto ne tiene: si parte dai token

«La riunione è stata rinviata a causa dello sciopero dei trasporti.»
11 parole, 66 caratteri

«The meeting has been postponed because of the transport strike.»
10 parole, 63 caratteri

Un tokenizer taglia il testo in pezzi, i token. In quanti pezzi taglia la frase italiana? Più o meno delle sue 11 parole? E dei suoi 66 caratteri?

La risposta è nella pagina che segue, con un tokenizer pubblico; quanti token pesa la frase per Claude lo misuriamo dal vivo tre pagine più avanti, con la demo M1-D02.

La frase, spezzata

Un tokenizer taglia il testo in token: venti pezzi, e non sempre sono parole

Tokenizer

Testo:

La riunione è stata rinviata a causa dello sciopero dei trasporti.

Token (tokenizer OpenAI, o200k): **20** di 20

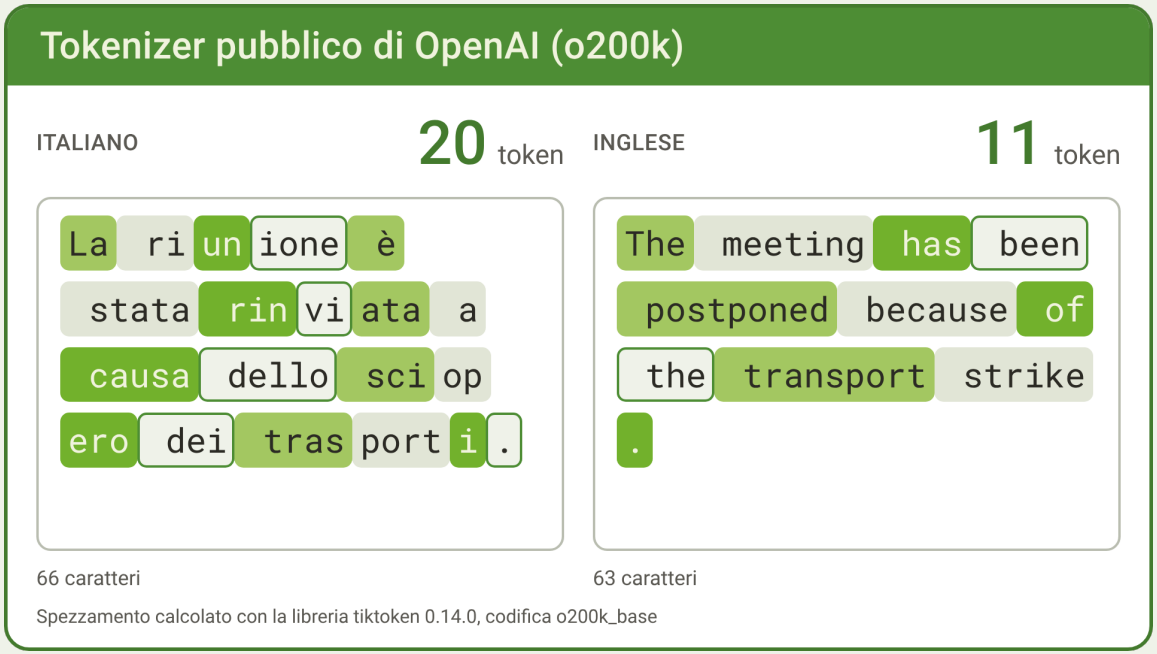


I venti pezzi della frase secondo il tokenizer pubblico di OpenAI (o200k)
Il tokenizer di Claude non è pubblico: un altro tokenizer, un altro numero
© Simone Scannapieco — spezzamento del tokenizer o200k, con la libreria tiktoken

- ✔ La risposta: **venti pezzi** — più delle 11 parole, molto meno dei 66 caratteri: un token **non** è una parola, e nemmeno una lettera
- Pezzi di parola: `ri` | un | `ione`, `sci` | op | `ero` — mentre «causa» e «dello» restano intere
- Il modello lavora su questi pezzi, **non sulle lettere** che ci sono dentro

Stessa frase, due lingue

Con il tokenizer pubblico di OpenAI (o200k): l’italiano costa più token dell’inglese



La stessa frase, in italiano e in inglese: 20 token contro 11
Spezzamenti del tokenizer pubblico di OpenAI (o200k), non di quello di Claude
© Simone Scannapieco — spezzamento del tokenizer o200k, con la libreria tiktoken

- Su questa frase, con questo tokenizer, l’italiano usa l’**82 % di token in più**
- In media, su duemila frasi tradotte da persone, un altro tokenizer di OpenAI, cl100k (ChatGPT e GPT-4), usa **circa 1,6 volte** i token dell’inglese per dire la stessa cosa in italiano (Petrov et al., NeurIPS 2023)
- E Claude? Il suo tokenizer non è pubblico: i suoi token li misuriamo **dal vivo**, con M1-D02, nella slide che segue

Stessa frase, due lingue

M1-D02, dal vivo: i token di Claude, letti nella risposta di Claude Code

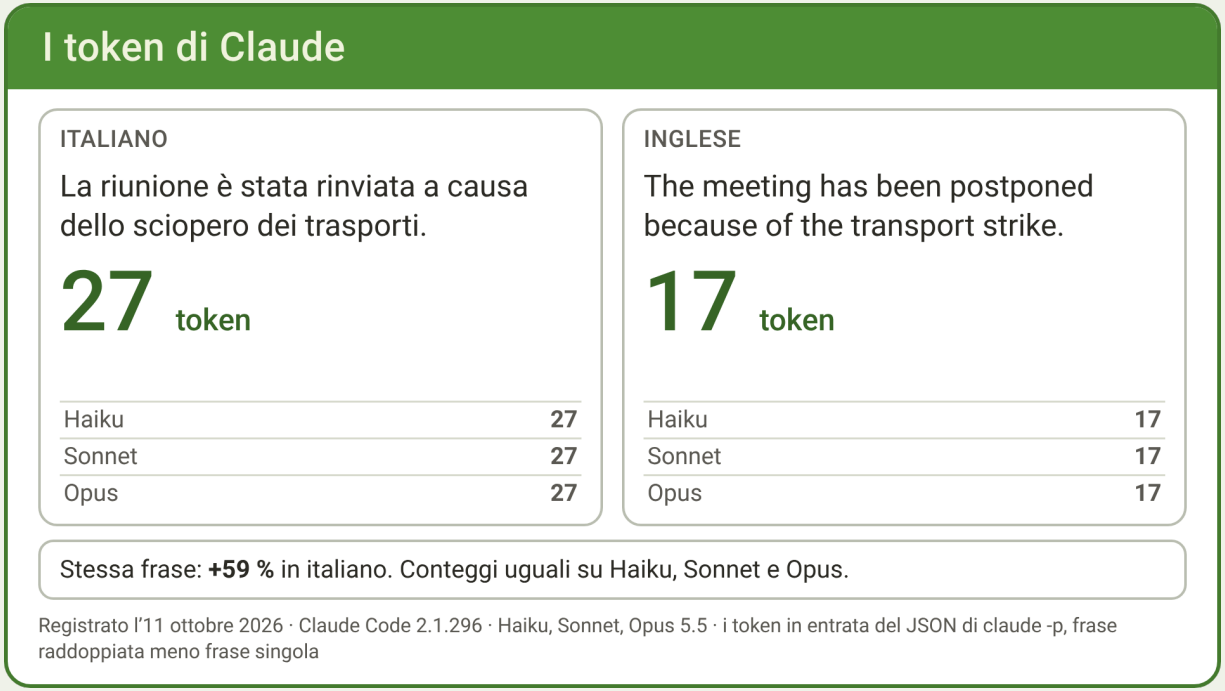
- Quattro chiamate a Claude Code, su Haiku: la frase italiana, la stessa frase **due volte**, poi l'inglese, singola e doppia
- Guardate i **token in entrata** di ogni risposta: oltre alla frase il modello riceve sempre altro testo, come l'istruzione di sistema — la **differenza** fra frase doppia e frase singola è il peso della frase

```
claude -p "$(cat frase_it.txt)" --model haiku --effort low \
  --tools "" --system-prompt "$(cat istruzione.txt)" \
  --strict-mcp-config --no-session-persistence \
  --output-format json | python3 leggi_json.py token_entrata
```

- È la prima delle quattro chiamate, nello spazio di lavoro di `tools/prepara_esercizio.sh M1-D02`: i passi completi sono nel README dell'elemento, nel repository delle esercitazioni

Stessa frase, due lingue

M1-D02, dal vivo: i token di Claude, letti nella risposta di Claude Code

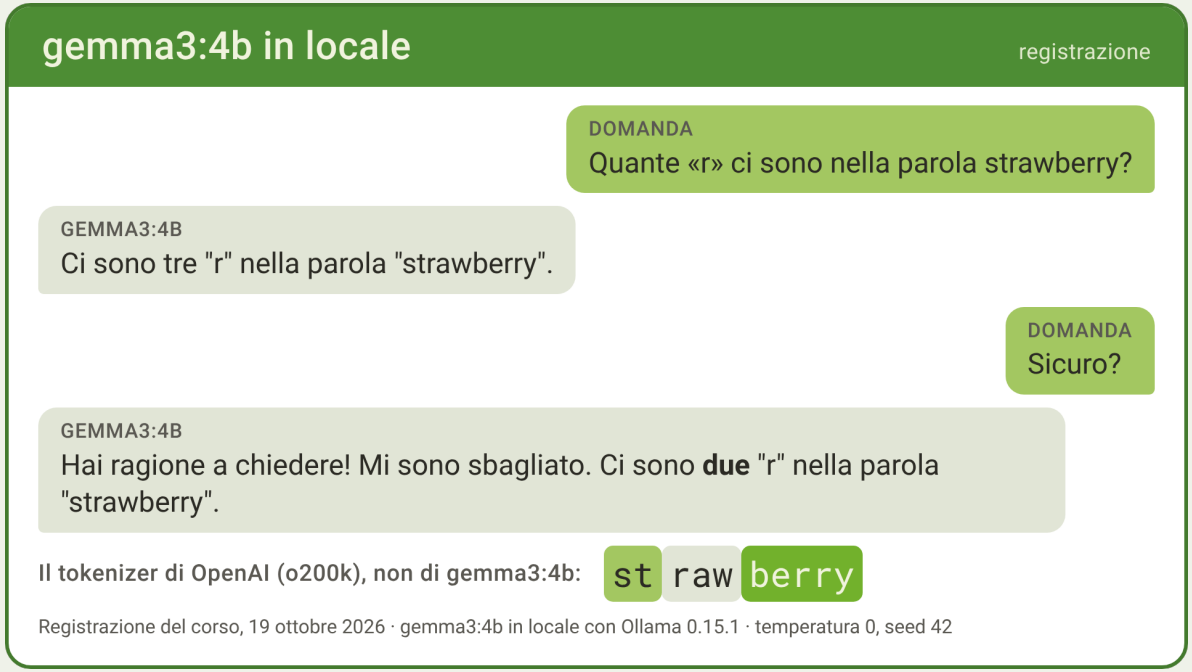


La stessa frase in italiano e in inglese: i token in entrata di Claude, frase doppia meno frase singola
Registrazione del corso, 11 ottobre 2026: stessi conteggi su Haiku, Sonnet e Opus
© Simone Scannapieco — figura dalla registrazione

- **Anthropic**, glossario: «per Claude, il numero di caratteri rappresentato da un token dipende dal modello e dalla lingua». Il controllo, sulla registrazione: **2,4 caratteri per token** in italiano, 3,7 in inglese — **confermata** su tutti e tre i modelli
- Su questa frase l'italiano costa a Claude il **59 % di token in più** (con il tokenizer di OpenAI l'82 %): **più consumo** del vostro limite d'uso e **meno spazio** nella finestra di contesto, tutto il testo che il modello consulta mentre risponde

Strawberry

Il caso famoso: quante «r» in «strawberry»?



Quante «r» in «strawberry»: tre, poi due, dopo un «Sicuro?»
Registrazione del corso, 19 ottobre 2026: gemma3:4b in locale con Ollama, non Claude
© Simone Scannapieco — spezzamento misurato con il tokenizer pubblico di OpenAI (o200k)

- ✔ Le «r» sono **tre**. Qui sopra un modello aperto, in locale, risponde giusto — poi, a un solo «Sicuro?», **cambia idea** e sbaglia, sicuro di sé: gli basta un dubbio
- C'è stato un tempo in cui la maggior parte dei modelli sbagliava questo conteggio: un tutorial dell'Academy di Anthropic lo racconta **al passato**. Con il tokenizer di OpenAI la parola è **tre pezzi**, `st` | `raw` | `berry`: nessuno è una lettera
- E Claude? Il suo tokenizer non è pubblico, ma quanti token pesa «strawberry» per Claude **si misura**: è la demo della slide che segue

Contare le lettere

M1-D03: quanti token pesa una parola? Prima rispondete voi, poi lo misuriamo

- **A voce**: per Claude, «interconnessione» — **16 lettere** — pesa più o meno di 16 token? E scritta lettera per lettera, con gli spazi in mezzo?
- Demo **M1-D03**, dal vivo: sei chiamate a Claude Code, su Haiku — «interconnessione», la stessa parola compitata e «strawberry», ciascuna da sola e **due volte**
- Come in M1-D02, il **peso** di un testo è la differenza fra i token in entrata del testo doppio e quelli del testo da solo

```
claude -p "$(cat parola.txt)" --model haiku --effort low \
  --tools "" --system-prompt "$(cat istruzione.txt)" \
  --strict-mcp-config --no-session-persistence \
  --output-format json | python3 leggi_json.py token_entrata
```

- È la prima delle sei chiamate, nello spazio di lavoro di `tools/prepara_esercizio.sh M1-D03`: i passi completi, e il controllo `controlla_pesi.py`, sono nel README dell'elemento

Contare le lettere

M1-D03: quanti token pesa una parola? Prima rispondete voi, poi lo misuriamo

Quanti token pesa una parola

IL PESO DI UN TESTO: I TOKEN IN ENTRATA DEL TESTO RIPETUTO DUE VOLTE MENO QUELLI DEL TESTO DA SOLO · IN VERDE SE TORNA CON L'AFFERMAZIONE DELLA SUA RIGA

	lettere	Haiku	Sonnet	Opus	affermazione
«interconnessione»	16	5	5	5	1
«i n t e r c o n n e s s i o n e»	16	16	16	16	2
«strawberry»	10	4	4	4	1

1. Ogni parola pesa meno token delle sue lettere: **confermata**
2. La parola compitata pesa più token della parola intera: **confermata**
3. Gli stessi pesi su ogni modello: **confermata**

Registrato il 20 ottobre 2026 · Claude Code 2.1.305 · Haiku, Sonnet, Opus 5.5 · claude -p, nessuno strumento, effort low · pesi e verdetti: demo/controlla_pesi.py

I pesi in token dei tre testi, per modello, e i verdetti del controllo dell'elemento
Registrazione del corso, 20 ottobre 2026: Haiku, Sonnet e Opus
© Simone Scannapieco — figura dalla registrazione

- **Anthropic**: il testo «viene codificato in una serie di token», e i token più grandi «vengono utilizzati quando possibile» (glossario); «un numero per pezzo di testo», non per carattere (Academy, traduzione nostra); dai modelli 4.7 in poi, «un tokenizer più recente» (pagina dei prezzi)
- Registrazione: «interconnessione», 16 lettere, **5 token**; compitata, **16**; «strawberry», 10 lettere, **4**, gli stessi pesi su Haiku, Sonnet e Opus 5.5 — le tre affermazioni del controllo **confermate**: le prime due su ciascun modello, la terza confrontando i tre
- Per chi scrive codice, le lettere le conta il codice:
  `grep -o n <<< interconnessione | wc -l` stampa `4`

Token, costi e limiti

Il metro di tutto il resto del corso

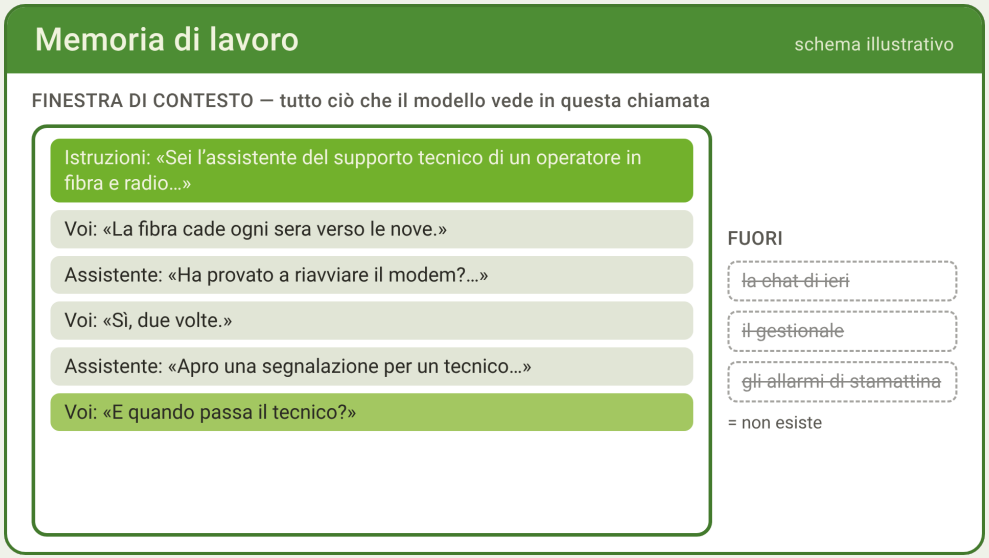
- Sull'API si paga **per milione di token**, in entrata e in uscita: l'uscita costa **5 volte** l'entrata, su tutti i modelli
- Fra modelli, in rapporti: rispetto a Haiku, Sonnet costa **20 volte** e Opus **40 volte** (il prezzo di Haiku vale per prompt fino a 100 mila token)
- Sul vostro abbonamento i token consumano il **limite d'uso**, e a ritmi diversi: Haiku il più leggero, Sonnet moderato, Opus pesante
- E il metro cambia con il modello: dai modelli Claude 4.7 in poi un tokenizer più recente dà circa il **30 % di token in più** per lo stesso testo

Il modello non vede lettere né parole: vede token — e i token sono il metro di costi e limiti.

I listini e il piano Pro, con i conti, nel Modulo 8.

La RAM del modello

La finestra di contesto: memoria di lavoro, non memoria permanente

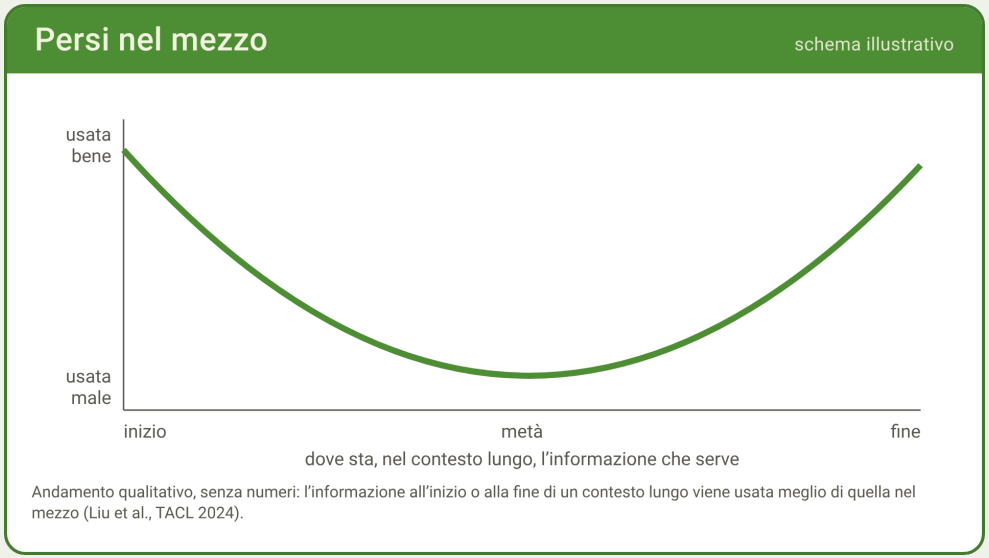


La finestra di contesto: tutto ciò che il modello vede in una chiamata, e ciò che ne resta fuori
Schema illustrativo: i testi sono inventati
© Simone Scannapieco

- È tutto il testo che il modello consulta mentre risponde, risposta compresa: la sua **memoria di lavoro**, distinta da ciò su cui è stato addestrato
- Ciò che è fuori, per il modello **non esiste**: per questo non impara dalle vostre correzioni, e la memoria di claude.ai, attiva di serie sul Pro, è **altro testo** rimesso nella finestra

Perché le chat lunghe perdono il filo

Vi è mai capitato? E che cosa avete fatto? Ditelo a voce



Dove sta, in un contesto lungo, l'informazione che serve, e quanto bene viene usata: una U

Schema illustrativo: l'andamento, non i dati dell'articolo

da Liu et al., «Lost in the Middle», TACL 2024

- Più token, meno precisione e meno richiamo: Anthropic chiama questo degrado **context rot**
- E conta **dove**: all'inizio e alla fine l'informazione si usa meglio che **nel mezzo**, anche nei modelli fatti per i contesti lunghi
- Rimedi: **in testa** ciò che conta; i testi lunghi **a pezzi**; le informazioni critiche **ripetute** — un log di migliaia di righe si filtra prima, non si incolla intero

Ogni chiamata riparte da zero

M1-D04, dal vivo: tre chiamate a Haiku in Claude Code

- **A**, in una sessione nuova: la linea LN-000204 e il suo guasto. **B**, in un'altra sessione nuova: «Quale linea seguo, e che guasto ha?». **C**: la stessa domanda nella sessione di A, **ripresa**
- Guardate che cosa rispondono B e C, e i **token in entrata** di A e di C

```
chiedi() {
  claude -p "$(cat "$1")" --model haiku --effort low --tools "" \
    --system-prompt "$(cat istruzione.txt)" --strict-mcp-config \
    --output-format json "${@:2}"
}
a="$(chiedi primo.txt)"
b="$(chiedi domanda.txt --no-session-persistence)"
c="$(chiedi domanda.txt --resume \
  "$(python3 leggi_json.py session_id <<< "$a")")"
```

Ogni chiamata riparte da zero

M1-D04, dal vivo: tre chiamate a Haiku in Claude Code



Tre chiamate a Haiku: due sessioni nuove, poi la prima ripresa (la risposta di B è tagliata: […])
Registrazione del corso, 11 ottobre 2026; linea e guasto inventati
© Simone Scannapieco — figura dalla registrazione

- **Anthropic**: «La Messages API è stateless (senza stato), il che significa che invii sempre l'intera cronologia della conversazione all'API» (guida «Utilizzo della Messages API»)
- Il controllo, sulla registrazione: B non nomina la linea, e C manda più token in entrata di A, **660 contro 608** — le due garanzie **confermate**; C la nomina, perché Claude Code le ha **rimandato** la conversazione di A. In una chat lunga, ogni turno rimanda tutta la finestra

Il modello non si ricorda di voi: ricorda solo ciò che gli rimandate ogni volta.

Ordini di grandezza

Fotografia di ottobre 2026: le finestre crescono, i nomi cambiano

Finestra	Chi, oggi
1 milione, nell’API	Claude Fable, Opus, Sonnet e Haiku di oggi; in uscita fino a 128 mila token
1 milione, sul Pro	Opus, Sonnet e Haiku 5.5, in claude.ai e in Claude Code
500 mila, in claude.ai	alcuni modelli precedenti, per esempio Opus 4.8 e Sonnet 4.6
200 mila, in claude.ai	gli altri modelli: circa 500 pagine di testo
circa 1 milione	per confronto, dalle pagine dei produttori: GPT-6 Astra, Gemini 3.8 Flash

- Un milione di token, sul tokenizer attuale: circa **555.000 parole**, 2,5 milioni di caratteri
- «Ci sta» non vuol dire «lo usa bene»: la U di prima vale anche qui — **un contesto curato batte una finestra piena**